TSwap Protocol Audit Report

Version 1.0

FC-Pascal

April 6, 2026

Protocol Audit Report

FC-Pascal

April 6, 2026

Prepared by: FC-Pascal Lead Auditors: - FC-Pascal

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    * [H-1] `TSwapPool::deposit` is missing deadline check causing transaction to execute even after the deadline
    * [H-2] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to charge users way too much, resulting in lost fees.
    * [H-3] Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens
    * [H-4] `TswapPool::sellPoolTokens` mismatches input and output tokens causing users to receive the incorrect amount of tokens

    * [H-5] In `TSwapPool::_swap` the extra tokens given to users after every swapCount breaks the protocol invariant of `x * y = k`
  - Low
    * [L-1] `TSwapPool::LiquidityAdded` has events out of order
    * [L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value
    * [S-#] TITLE (Root + Impact)
  - Informational
    * [I-1] `PoolFactory::PoolFactory__PoolAlreadyExists` is not used and should be removed
    * [I-1] Lacking zero adddress checks
    * [I-3] `PoolFactory::createPool` should use `.symbol()` and not `.name()`
    * [I-4] Three events should be indexed if there are more than 3 parameters
    * [I-5] Lacking zero adddress checks

Protocol Summary

Protocol does X, Y, Z

Disclaimer

The YOUR_NAME_HERE team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

	Impact		
	High	Medium	Low
High	H	H/M	M

		Impact		
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

Roles

Executive Summary

Issues found

Severity	Number of issues found
High	4
Medium	2
Low	2
Info	9
Total	17

Findings

High

[H-1] `TSwapPool::deposit` is missing deadline check causing transaction to execute even after the deadline

Description The `deposit` has a deadline parameter after which the transaction does not execute again, but this parameter is never used. As a consequence, the the operations that add liquidity to the pool might be executed at unexpected times, in market conditions where the market condition is unfavourable.

Impact Transactions could be sent when market conditions are unfavorable to deposit, even after adding a deadline parameter.

Proof of Concepts The `deadline` parameter is used.

Recommended mitigation Recommend making the following changes to the fucntion:

```
    function deposit(
        uint256 wethToDeposit,
        uint256 minimumLiquidityTokensToMint,
        uint256 maximumPoolTokensToDeposit,
        // @audit-high deadline never used
        uint64 deadline
    )
        external
+       revertIfDeadlinePassed(deadline)
        revertIfZero(wethToDeposit)
        returns (uint256 liquidityTokensToMint)
```

[H-2] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to charge users way too much, resulting in lost fees.

Description The `getInputAmountBasedOnOutput` should calculate the amount of tokens a user should deposit based on amount of output tokens but the function miscalculates. The amount is being scaled by 10_000 instead of 1_000

Impact Protocol takes more fees than expected from the user

Recommended mitigation :

```
    function getInputAmountBasedOnOutput(
        uint256 outputAmount,
        uint256 inputReserves,
```

```
        uint256 outputReserves
    )
        public
        pure
        revertIfZero(outputAmount)
        revertIfZero(outputReserves)
        returns (uint256 inputAmount)
    {

        return
-           ((inputReserves * outputAmount) * 10000) /
-           ((outputReserves - outputAmount) * 997);
+           ((inputReserves * outputAmount) * 10000) /
+           ((outputReserves - outputAmount) * 997);
    }
```

[H-3] Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens

Description: The `swapExactOutput` function does not include any slippage protection

Impact: If market conditions change before the swap executes, the user could get a much worse swap

Proof of Concepts 1. The price of 1 WETH right now is 1,000 USDC 2. User inputs a swapExactOutput looking for 1 WETH 1. inputToken = USDC 2. outputToken = WETH 3. outputAmount = 1 4. deadline = whatever 3. The function does not offer a maxInput amount 4. As the transaction is pending in the mempool, the market changes! And the price moves HUGE -> 1 WETH is now 10,000 USDC. 10x more than the user expected 5. The transaction completes, but the user sent the protocol 10,000 USDC instead of the expected 1,000 USDC

Recommended mitigation: Include a `maxInputAmount` so that user only spends up to a specific amoun, and can predict how much they will spend on the protocol.

```
+     function swapExactOutput(
+        IERC20 inputToken,
+        uint256 maxInputAmount,
.
.
.
    inputAmount = getInputAmountBasedOnOutput(
            outputAmount,
            inputReserves,
            outputReserves
        );
+        if (inputAmount > maxInputAmount) {
+            revert();
```

```
+        }

        _swap(inputToken, inputAmount, outputToken, outputAmount);
```

[H-4] `TswapPool::sellPoolTokens` mismatches input and output tokens causing users to receive the incorrect amount of tokens

Description: The `sellPoolTokens` function is intended to allow users to easily sell pool tokens and receive WETH in exchange. Users indicate how many pool tokens they're willing to sell in the `poolTokenAmount` parameter. However, the function currently miscalculaes the swapped amount.

This is due to the fact that the `swapExactOutput` function is called, whereas the `swapExactInput` function is the one that should be called. Because users specify the exact amount of input tokens, not output.

Impact: Users will swap the wrong amount of tokens, which is a severe disruption of protcol functionality.

Proof of Concepts:

Recommended mitigation: Consider changing the implementation to use `swapExactInput` instead of `swapExactOutput`. Note that this would also require changing the `sellPoolTokens` function to accept a new parameter (ie `minWethToReceive` to be passed to `swapExactInput`)

```
        function sellPoolTokens(****
        uint256 poolTokenAmount,
+       uint256 minWethToReceive,
        ) external returns (uint256 wethAmount) {
-        return swapExactOutput(i_poolToken, i_wethToken, poolTokenAmount, uint64(block.timestamp));
+        return swapExactInput(i_poolToken, poolTokenAmount, i_wethToken, minWethToReceive, uint64(block.timestamp));
    }
```

[H-5] In `TSwapPool::_swap` the extra tokens given to users after every swapCount breaks the protocol invariant of `x * y = k`

Description: The protocol follows a strict invariant of `x * y = k`. Where: 1. `x`: The balance of the pool token 2. `y`: The balance of WETH 3. `k`: The constant product of the two balances

This means, that whenever the balances change in the protocol, the ratio between the two amounts should remain constant, hence the k. However, this is broken due to the extra incentive in the _swap function. Meaning that over time the protocol funds will be drained.

The follow block of code is responsible for the issue.

```
            swap_count++;
        if (swap_count >= SWAP_COUNT_MAX) {
            swap_count = 0;
            outputToken.safeTransfer(msg.sender, 1
                _000_000_000_000_000_000);
        }
```

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentive given out by the protocol.

Proof of Concepts: 1. A user swaps 10 times, and collects the extra incentive of 1_000_000_000_000_000_000 tokens 2. That user continues to swap untill all the protocol funds are drained

Proof of Code

Place the following into `TSwapPool.t.sol`.

```
    function testInvariantBroken() public {
        vm.startPrank(liquidityProvider);
        weth.approve(address(pool), 100e18);
        poolToken.approve(address(pool), 100e18);
        pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
        vm.stopPrank();

        uint256 outputWeth = 1e17;

        vm.startPrank(user);
        poolToken.approve(address(pool), type(uint64).max);
        poolToken.mint(user, 100e18);
        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );
        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );
        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );
        pool.swapExactOutput(
            poolToken,
```

```
            weth,
            outputWeth,
            uint64(block.timestamp)
        );
        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );
        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );
        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );
        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );
        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        int256 startingY = int256(weth.balanceOf(address(pool)));
        int256 expectedDeltaY = int256(-1) * int256(outputWeth);

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );
        vm.stopPrank();



        uint256 endingY = weth.balanceOf(address(pool));
        int256 actualDeltaY = int256(endingY) - int256(startingY);
        assertEq(actualDeltaY, expectedDeltaY);
```

```

    }
```

Recommended mitigation: Remove the extra incentive mechanism. If you want to keep this in, we should account for the change in the `x * y = k` protocol invariant. Or, we should set aside tokens in the same way we do with fees.

```
-        swap_count++;
-        // Fee-on-transfer
-        if (swap_count >= SWAP_COUNT_MAX) {
-            swap_count = 0;
-            outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
-        }
```

Low

[L-1] `TSwapPool::LiquidityAdded` has events out of order

Description When the `LiquidityAdded` event is emitted in the `TSwapPool::_addLiquidityMintAndTrans` function, it logs events in a wrong order.

Impact Event emissions is incorrect leading to off-chain funtions potentially malfunctioning

Proof of Concepts

Recommended mitigation The event emission should be rewritten thus:

```
-   emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
+    emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);
```

[L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value

Description: The `swapExactInput` function is expected to return the actual amount of tokens bought by the caller. However, while it declares the named return value `output`, it is never assigned a value, nor uses an explicit return statement.

Impact: The return value will always be 0, giving an incorrect information to the caller.

Recommended mitigation:

```
    {
        uint256 inputReserves = inputToken.balanceOf(address(this));
        uint256 outputReserves = outputToken.balanceOf(address(this));

-        uint256 outputAmount = getOutputAmountBasedOnInput(
-            inputAmount,
-            inputReserves,
-            outputReserves
-        );

+        uint256 output = getOutputAmountBasedOnInput(
+            inputAmount,
+            inputReserves,
+            outputReserves
+        );

-        if (output < minOutputAmount) {
-            revert TSwapPool__OutputTooLow(output, minOutputAmount);
-        }

-        _swap(inputToken, inputAmount, outputToken, output);

+        if (output < minOutputAmount) {
+            revert TSwapPool__OutputTooLow(output, minOutputAmount);
+        }

+        _swap(inputToken, inputAmount, outputToken, output);
    }
```

[S-#] TITLE (Root + Impact)

Description

Impact

Proof of Concepts

Recommended mitigation

Informational

[I-1] `PoolFactory::PoolFactory__PoolAlreadyExists` is not used and should be removed

[I-1] Lacking zero adddress checks

```
constructor(address wethToken) {
        if (wethToken == address(0)) {
            revert();
        }
        i_wethToken = wethToken;
    }
```

[I-3] `PoolFactory::createPool` should use `.symbol()` and not `.name()`

```
-       string memory liquidityTokenSymbol = string.concat("ts", IERC20(tokenAddress).name());
+       string memory liquidityTokenSymbol = string.concat("ts", IERC20(tokenAddress).symbol());
```

[I-4] Three events should be indexed if there are more than 3 parameters

```
-    event Swap(
        address indexed swapper,
        IERC20 tokenIn,
        uint256 amountTokenIn,
        IERC20 tokenOut,
        uint256 amountTokenOut
    );
+    event Swap(
        address indexed swapper,
        IERC20 indexed tokenIn,
        uint256 indexed amountTokenIn,
        IERC20 tokenOut,
        uint256 amountTokenOut
    );
```

[I-5] Lacking zero adddress checks

```
    constructor(
        address poolToken,
        address wethToken,
        string memory liquidityTokenName,
        string memory liquidityTokenSymbol
    ) ERC20(liquidityTokenName, liquidityTokenSymbol) {
        if (wethToken == address(0)) {
            revert();
        }
        i_wethToken = IERC20(wethToken);
        i_poolToken = IERC20(poolToken);
```

```
    }
```